

РЕФЕРАТ

Рассчетно-пояснительная записка содержит 63 страницы, 14 иллюстраций, 15 таблиц, 17 источников, 3 приложения.

Ключевые слова: спортивный зал, клиент, тренер, база данных, ролевая модель, система управления базами данных, TypeScript, PostgreSQL, индекс.

Темой данной курсовой работы является исследование предметной области спортивных залов и комплексов. Целью является разработка программного обеспечения с использованием базы данных, реализующего управление спортивным залом. Результатами работы является программный продукт на языке TypeScript, использующий систему управления базами данных PostgreSQL, содержащий сценарии использования для клиентов, тренеров и администраторов спортивного зала, а также исследование влияния применения различных типов индексов в базе данных на время выполнения запросов к ней.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 Аналитическая часть	7
1.1 Анализ предметной области	7
1.2 Сравнительный анализ существующих конкурентов	7
1.3 Требования к разрабатываемой базе данных и приложению . . .	8
1.4 Формализация и описание информации, подлежащей хранению в проектируемой базе данных	8
1.5 Анализ существующих баз данных на основе формализации данных	10
1.5.1 Дореляционные базы данных	10
1.5.2 Реляционные базы данных	10
1.5.3 Постреляционные базы данных	10
1.5.4 Вывод	11
1.6 Формализация и описание пользователей проектируемого при- ложения к базе данных	11
1.7 Диаграмма прецедентов	12
1.8 Формализация задачи	13
2 Конструкторская часть	16
2.1 Описание сущностей проектируемой базы данных	16
2.2 Диаграмма проектируемой базы данных	19
2.3 Описание проектируемых ограничений целостности базы данных	21
2.4 Описание проектируемых функций в формате схемы	25
2.5 Описание проектируемой ролевой модели на уровне базы данных	28
3 Технологическая часть	30
3.1 Средства реализации	30
3.2 Структура классов	30
3.3 Выбор системы управления базами данных	31
3.4 Классы эквивалентности	33
3.4.1 Классы эквивалентности для функции get_coach_specializations	33

3.4.2	Классы эквивалентности для функции show_available_events	35
3.4.3	Классы эквивалентности для функции register_client_for_training	36
3.5	Разработанные функции	37
3.6	Разработанная ролевая модель	40
3.7	Демонстрация работы программы	43
4	Исследовательская часть	46
4.1	Постановка исследования	46
4.2	Технические характеристики	46
4.3	План исследования	46
4.4	Результаты исследования	48
4.5	Вывод	52
	ЗАКЛЮЧЕНИЕ	53
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	55
	ПРИЛОЖЕНИЕ А	56
	ПРИЛОЖЕНИЕ Б Сценарий создания базы данных	57
	ПРИЛОЖЕНИЕ В Сценарий наложения ограничений на таблицы базы данных	60

ВВЕДЕНИЕ

Спортивным залам необходимо управлять внутренними бизнес процессами. Для эффективного решения этой задачи требуется программное обеспечение, облегчающее контроль и управление залом. Оно повысит качество обслуживания и оптимизирует работу зала, сделав ее более удобной для администраторов, тренеров и клиентов.

Цель данной работы — разработка и реализация информационной системы, прикладные программы которой используют данные, хранящиеся в базе данных, для автоматизации деятельности спортивного зала.

Для достижения данной цели необходимо решить следующие задачи:

- уточнить требования и формулировку ограничений предметной области;
- проанализировать существующие решения;
- спроектировать базу данных;
- спроектировать способы взаимодействия с базой данных;
- разработать базу данных и приложение доступа к базе данных;
- провести исследование разработанной базы данных для определения эффективности индексации.

1 Аналитическая часть

1.1 Анализ предметной области

Спортивным залом зачастую называется целый спортивный комплекс, имеющий множество специализированных пространств для проведения тренировок. Такое предприятие организует множество людей, например тренеров, клиентов, администраторов. Необходимо регулировать назначение тренировок, запись клиентов, штат администраторов и тренеров, ведение персональных статистик, составление расписания залов и спортивных мероприятий.

1.2 Сравнительный анализ существующих конкурентов

Для анализа были выбраны популярные приложения с высокими рейтингами, такие как:

- 1С:Фитнес-клуб [1];
- Mobifitness [2];
- FitBase [3];
- Sport Priority [4].

При анализе будут учитываться следующие критерии:

1. управление персоналом;
2. учёт клиентов;
3. управление графиком персонала;
4. возможность ведения клиентом персональной статистики;
5. управление спортивными пространствами;
6. управление тренировками.

В таблице 1.1 приведены результаты анализа.

Таблица 1.1 – Результаты анализа существующих конкурентов

Номер критерия	1С:Фитнес-клуб	Mobifitness	FitBase	Sport Priority
1	+	+	+	+
2	+	+	+	+
3	+	-	-	+
4	-	+	-	-
5	+	+	-	-
6	+	+	+	+

1.3 Требования к разрабатываемой базе данных и приложению

База данных должна включать в себя данные о клиентах, администраторах, тренерах, тренировках, спортивных пространствах.

Приложение должно позволять выполнять следующие операции:

1. авторизовываться как клиент, тренер, администратор;
2. управлять персональными статистиками клиентов;
3. управлять расписаниями спортивных пространств;
4. управлять персоналом и клиентами;
5. управлять тренерскими специализациями;
6. управлять рабочими часами тренера;
7. управлять групповыми мероприятиями и тренировками;
8. управлять персональными тренировками.

1.4 Формализация и описание информации, подлежащей хранению в проектируемой базе данных

В таблице 1.2 описана информация, подлежащая хранению в базе данных.

Таблица 1.2 – Категории данных в базе данных и их свойства

Категория	Информация
Тренер	идентификатор, имя, фамилия, логин, пароль, статус(уволен, работает)
Вид тренерской специализации	идентификатор, название, идентификатор зала проведения
Связь тренеров и видов специализаций	идентификатор, идентификатор тренера, идентификатор специализации
Рабочие часы тренера	идентификатор, идентификатор тренера, дата, время начала, время окончания
Клиент	идентификатор, имя, фамилия, логин, пароль, статус(посещает зал, перестал ходить)
Виды персональных статистик	идентификатор, название, единица измерения
Связь статистик клиентов	идентификатор, идентификатор клиента, идентификатор статистики, значение, дата
Администратор	идентификатор, имя, фамилия, логин, пароль, статус(уволен, работает)
Групповое мероприятие	идентификатор, название, идентификатор назначившего администратора, идентификатор тренера, идентификатор спортивного пространства, вместимость, продолжительность, дата и время
Спортивное пространство	идентификатор, название
Расписание работы спортивного пространства	идентификатор, идентификатор спортивного пространства, дата, время начала, время окончания
Записи на мероприятия	идентификатор, идентификатор клиента, идентификатор группового мероприятия
Индивидуальные тренировки	идентификатор, идентификатор тренера, идентификатор клиента, идентификатор спортивного пространства, дата и время, продолжительность, дополнительная информация

1.5 Анализ существующих баз данных на основе формализации данных

Существует три основных типа баз данных:

1. дореляционные;
2. реляционные;
3. постреляционные.

1.5.1 Дореляционные базы данных

Дореляционные базы данных [5] хранят записи в виде связей родитель-ребёнок. Основными моделями таких баз являются иерархическая и сетевая. Основное отличие между ними заключается в том, что иерархическая модель не поддерживает отношение многие-ко-многим, так как каждый ребёнок может иметь только одного родителя. В свою очередь в сетевой модели у каждого ребёнка может быть несколько родителей.

Основными недостатками дореляционных баз данных являются избыточность данных и сложность выполнения запросов. Масштабируемость затруднена из-за сложности изменения структуры, гибкость крайне ограничена. Целостность данных поддерживается, но за счёт жёсткой структуры связей.

1.5.2 Реляционные базы данных

В реляционных базах данных данные хранятся в виде двумерных таблиц. Связь между ними обеспечивается общими столбцами, такими как идентификаторы. Табличная архитектура реализует эффективный способ хранения структурированной информации.

Они обеспечивают гибкий и масштабируемый способ хранения и управления большими объемами данных, гарантируя при этом целостность и непротиворечивость данных с помощью ограничений.

1.5.3 Постреляционные базы данных

Такие базы данных позволяют удобно хранить неструктурированные данные, поддерживая логические связи между ними. Постреляционные базы

отлично подходят для данных с изменчивой структурой и высоконагруженных приложений. Недостатком является сложность обеспечения целостности данных.

Отличаются высокой гибкостью структуры, хорошей горизонтальной масштабируемостью. Сложность запросов варьируется в зависимости от конкретной реализации.

1.5.4 Вывод

Для реализации поставленной задачи была выбрана реляционная модель, так как она лучше всего подходит для хранения структурированной информации, обеспечивая баланс между целостностью данных, гибкостью запросов и возможностями масштабирования.

1.6 Формализация и описание пользователей проектируемого приложения к базе данных

Гость, или неавторизованный пользователь может войти как клиент, тренер, или администратор.

Администратор может:

1. управлять видами индивидуальных статистик клиентов;
2. управлять расписанием работы спортивных пространств;
3. управлять клиентами, тренерами, и другими администраторами;
4. управлять специализациями тренеров;
5. управлять рабочими часами тренеров;
6. управлять групповыми мероприятиями;
7. просматривать проводимые тренировки тренеров.

Тренер может:

1. управлять своими рабочими часами;
2. управлять персональными тренировками;

3. просматривать предстоящие тренировки и мероприятия;
4. просматривать доступные залы для проведения тренировок в зависимости от его специализаций.

Клиент может:

1. управлять персональной статистикой;
2. просматривать доступные мероприятия;
3. управлять записями на мероприятия и тренировки;
4. отменять персональные тренировки.

1.7 Диаграмма прецедентов

На рисунке 1.1 представлена диаграмма прецедентов.

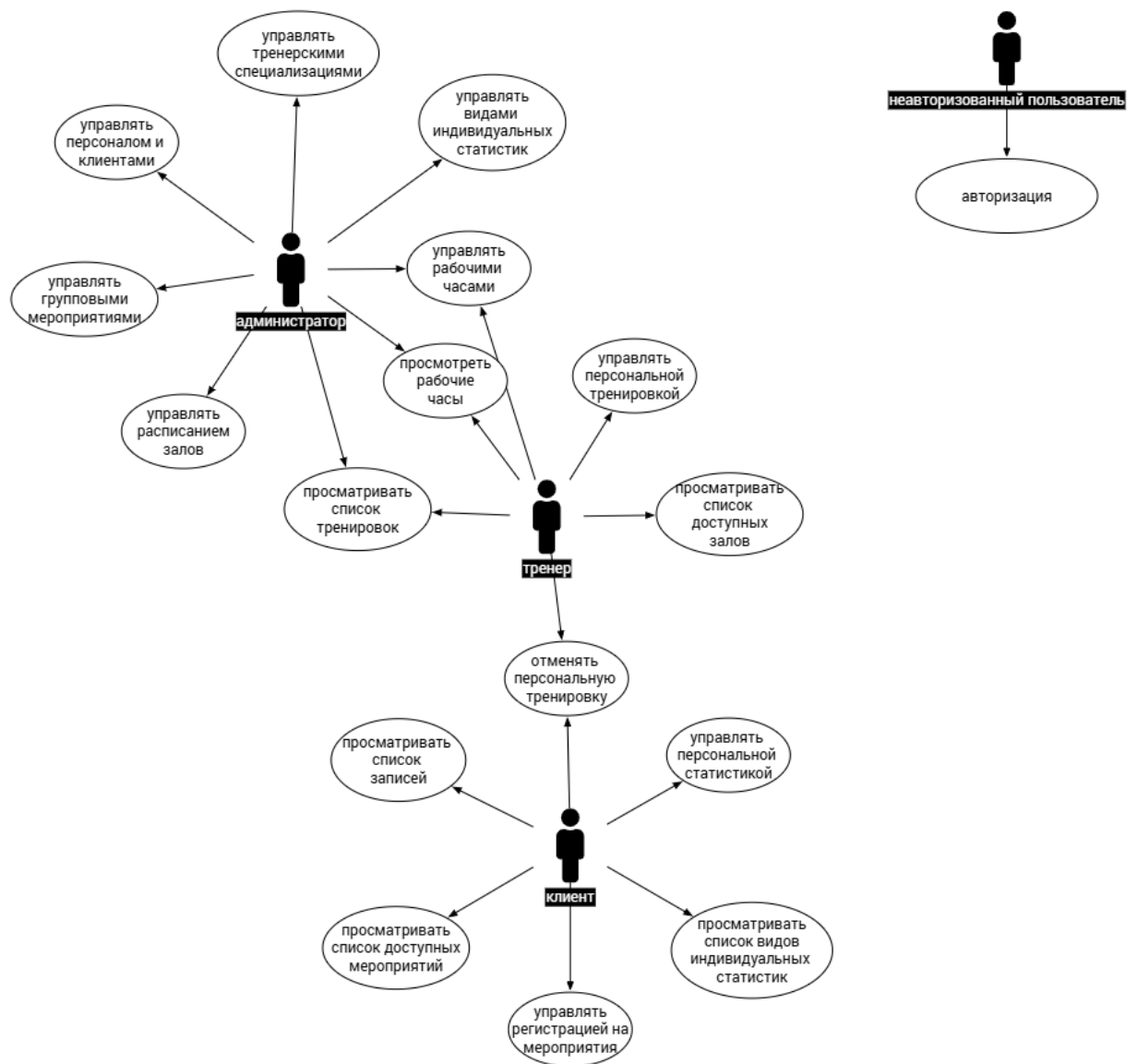


Рисунок 1.1 – Диаграмма прецедентов

1.8 Формализация задачи

На основании проведённого анализа предметной области, требований к разрабатываемой базе данных и приложению, сравнительного анализа предметной области, формализация задачи проектирования базы данных и приложения для спортивного зала звучит следующим образом:

Требуется создать информационную систему спортивного зала с использованием реляционной базы данных для управления:

1. клиентами;
2. тренерами;

3. администраторами;
4. графиком работы персонала;
5. спортивными пространствами;
6. групповыми и индивидуальными тренировками;
7. персональными клиентскими статистиками.

На рисунке 1.2 представлена диаграмма сущность-связь проектируемой базы данных.

2 Конструкторская часть

2.1 Описание сущностей проектируемой базы данных

База данных содержит 13 таблиц:

1. Таблица «Coach» хранит информацию о тренерах. Содержит следующие поля:
 - ID – первичный ключ;
 - name – имя;
 - surname – фамилия;
 - login – логин;
 - password – пароль в защищённом виде;
 - is_participate – статус тренера(уволен, работает).
2. Таблица «CoachSchedule» хранит информацию о рабочих часах тренера. Содержит следующие поля:
 - ID – первичный ключ;
 - CoachID – идентификатор тренера;
 - date – дата в формате ГГГГ-ММ-ДД;
 - start – время начала работы в формате ЧЧ:ММ:СС;
 - stop – время окончания работы в формате ЧЧ:ММ:СС.
3. Таблица «Hall» хранит информацию о спортивных пространствах. Содержит следующие поля:
 - ID – первичный ключ;
 - name – название.
4. Таблица «CoachSpecializationTypes» хранит информацию о видах тренерских специализаций. Содержит следующие поля:
 - ID – первичный ключ;

- name – название;
 - HallID – идентификатор спортивного пространства для проведения тренировок тренером с такой специализацией.
5. Таблица «CoachSpecialization» хранит информацию о тренерах и какие специализации у них есть. Содержит следующие поля:
- ID – первичный ключ;
 - SpecID – идентификатор типа специализации;
 - CoachID – идентификатор тренера.
6. Таблица «HallSchedule» хранит информацию о расписании работы спортивных пространств. Содержит следующие поля:
- ID – первичный ключ;
 - HallID – идентификатор спортивного пространства;
 - date – дата в формате ГГГГ-ММ-ДД;
 - start – время начала работы в формате ЧЧ:ММ:СС;
 - stop – время окончания работы в формате ЧЧ:ММ:СС.
7. Таблица «Admin» хранит информацию об администраторах. Содержит следующие поля:
- ID – первичный ключ;
 - name – имя;
 - surname – фамилия;
 - login – логин;
 - password – пароль в защищённом виде;
 - is_participate – статус администратора (уволен, работает).
8. Таблица «StandartTraining» хранит информацию о групповых тренировках и мероприятиях. Содержит следующие поля:
- ID – первичный ключ;
 - name – название;

- AdminID – идентификатор администратора, создавшего мероприятие;
- CoachID – идентификатор тренера;
- HallID – идентификатор спортивного пространства;
- capacity – количество мест на мероприятие;
- datetime – дата и время начала мероприятия в формате ГГГГ-ММ-ДД ЧЧ:ММ:СС;
- duration – продолжительность.

9. Таблица «Client» хранит информацию о клиентах. Содержит следующие поля:

- ID – первичный ключ;
- name – имя;
- surname – фамилия;
- login – логин;
- password – пароль в защищённом виде;
- is_participate – статус клиента(ходит, перестал ходить).

10. Таблица «ClientDynamicTypes» хранит информацию о возможных видах персональных клиентских статистик для отслеживания. Содержит следующие поля:

- ID – первичный ключ;
- name – название статистики;
- unit – мера измерения.

11. Таблица «ClientDynamic» хранит информацию о клиентах и их персональных статистиках. Содержит следующие поля:

- ID – первичный ключ;
- ClientID – идентификатор клиента;
- DynamicTypeID – идентификатор типа статистики;

- value – зафиксированное значение;
 - date – дата измерения в формате ГГГГ-ММ-ДД.
12. Таблица «TrainingSigns» хранит информацию о клиентах и их записях на групповые мероприятия и тренировки. Содержит следующие поля:
- ID – первичный ключ;
 - ClientID – идентификатор клиента;
 - StdTrainingID – идентификатор группового мероприятия.
13. Таблица «IndividualTraining» хранит информацию об индивидуальных тренировках. Содержит следующие поля:
- ID – первичный ключ;
 - CoachID – идентификатор тренера;
 - ClientID – идентификатор клиента;
 - HallID – идентификатор спортивного пространства;
 - datetime – дата и время начала тренировки в формате ГГГГ-ММ-ДД ЧЧ:ММ:СС;
 - duration – продолжительность;
 - notes – некоторые дополнительные заметки о тренировке.

2.2 Диаграмма проектируемой базы данных

На рисунке 2.1 представлена диаграмма проектируемой базы данных.

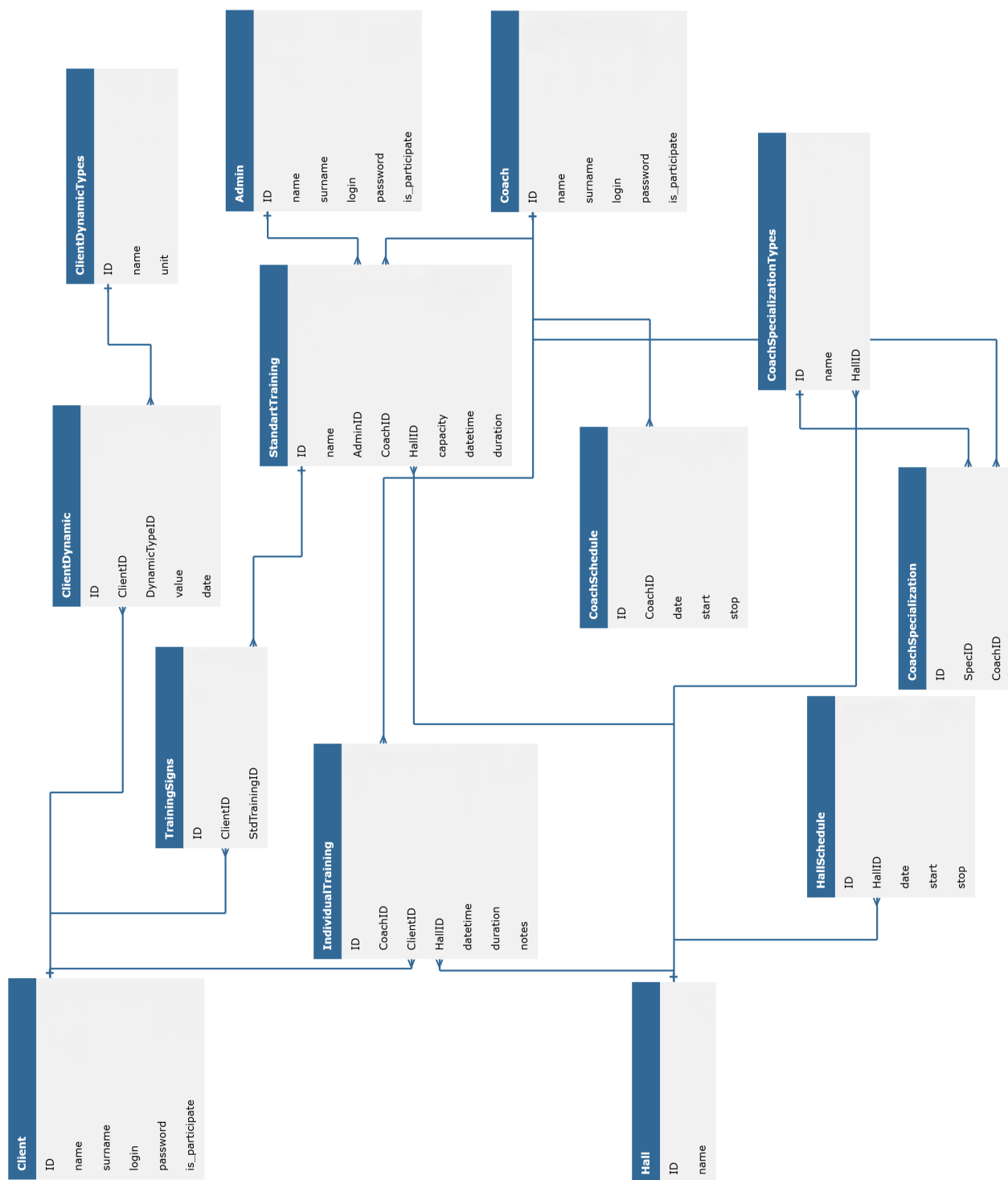


Рисунок 2.1 – Диаграмма проектируемой базы данных

2.3 Описание проектируемых ограничений целостности базы данных

1. Таблица Coach (Тренер):

- Поле name не может быть пустым;
- Поле name должно содержать только буквы (латинские или кириллические);
- Поле surname не может быть пустым;
- Поле surname должно содержать только буквы (латинские или кириллические);
- Поле login не может быть пустым;
- Поле login должно содержать только латинские буквы и цифры;
- Поле password не может быть пустым;
- Пароль должен быть длиной не менее 8 символов;
- Логин должен быть уникальным;
- Длина логина должна быть от 8 до 10 символов;
- Поле is_participate по умолчанию должно быть истинным.

2. Таблица CoachSpecializationTypes (Типы специализаций тренеров):

- Поле name не может быть пустым;
- Поле HallID является внешним ключом, ссылающимся на таблицу Hall;
- Поле name должно быть уникальным.

3. Таблица CoachSpecialization (Специализации тренеров):

- Поле SpecID является внешним ключом, ссылающимся на таблицу CoachSpecializationTypes;
- Поле CoachID является внешним ключом, ссылающимся на таблицу Coach;
- Пара SpecID, CoachID должна быть уникальной.

4. Таблица CoachSchedule (Расписание тренеров):

- Поле CoachID является внешним ключом, ссылающимся на таблицу Coach;
- Поле date не может быть пустым;
- Поле start не может быть пустым;
- Поле stop не может быть пустым;
- Комбинация CoachID и date должна быть уникальной.

5. Таблица Client (Клиент):

- Поле name не может быть пустым;
- Поле name должно содержать только буквы (латинские или кириллические);
- Поле surname не может быть пустым;
- Поле surname должно содержать только буквы (латинские или кириллические);
- Поле login не может быть пустым;
- Поле login должно содержать только латинские буквы и цифры;
- Поле password не может быть пустым;
- Пароль должен быть длиной не менее 8 символов;
- Логин должен быть уникальным;
- Длина логина должна быть от 8 до 10 символов;
- Поле is_participate по умолчанию должно быть истинным.

6. Таблица ClientDynamicTypes (Типы динамических данных клиента):

- Поле name не может быть пустым;
- Поле unit не может быть пустым;
- Поле name должно быть уникальным.

7. Таблица ClientDynamic (Динамические данные клиента):

- Поле ClientID является внешним ключом, ссылающимся на таблицу Client;
- Поле DynamicTypeID является внешним ключом, ссылающимся на таблицу ClientDynamicTypes;
- Поле value не может быть пустым;
- Поле date не может быть пустым;
- Тройка ClientID, DynamicTypeID, date должна быть уникальной.

8. Таблица Admin (Администратор):

- Поле name не может быть пустым;
- Поле name должно содержать только буквы (латинские или кириллические);
- Поле surname не может быть пустым;
- Поле surname должно содержать только буквы (латинские или кириллические);
- Поле login не может быть пустым;
- Поле login должно содержать только латинские буквы и цифры;
- Поле password не может быть пустым;
- Пароль должен быть длиной не менее 8 символов;
- Логин должен быть уникальным;
- Длина логина должна быть от 8 до 10 символов;
- Поле is_participate по умолчанию должно быть истинным.

9. Таблица StandartTraining (Групповые мероприятия):

- Поле name не может быть пустым;
- Поле name должно содержать только буквы (латинские или кириллические), может состоять из нескольких слов;
- Поле name не может состоять только из пробелов;
- Поле AdminID является внешним ключом, ссылающимся на таблицу Admin;

- Поле CoachID является внешним ключом, ссылающимся на таблицу Coach;
- Поле HallID является внешним ключом, ссылающимся на таблицу Hall;
- Поле capacity не может быть пустым;
- Вместимость должна быть больше 0;
- Поле datetime не может быть пустым;
- Поле duration не может быть пустым;
- Длительность должна быть больше 0.

10. Таблица Hall (Спортивное пространство):

- Поле name не может быть пустым;
- Поле name должно содержать буквы (латинские или кириллические), цифры и дефисы, может состоять из нескольких слов;
- Поле name не может состоять только из пробелов;
- Поле name должно быть уникальным.

11. Таблица HallSchedule (Расписание спортивного пространства):

- Поле HallID является внешним ключом, ссылающимся на таблицу Hall;
- Поле date не может быть пустым;
- Поле start не может быть пустым;
- Поле stop не может быть пустым;
- Комбинация HallID и date должна быть уникальной.

12. Таблица TrainingSigns (Записи на тренировки):

- Поле ClientID является внешним ключом, ссылающимся на таблицу Client;
- Поле StdTrainingID является внешним ключом, ссылающимся на таблицу StandartTraining;

- Пара ClientID, StdTrainingID должна быть уникальной.

13. Таблица IndividualTraining (Индивидуальные тренировки):

- Поле CoachID является внешним ключом, ссылающимся на таблицу Coach;
- Поле ClientID является внешним ключом, ссылающимся на таблицу Client;
- Поле HallID является внешним ключом, ссылающимся на таблицу Hall;
- Поле datetime не может быть пустым;
- Поле duration не может быть пустым;
- Длительность должна быть больше 0;
- Пара CoachID, datetime должна быть уникальной.

2.4 Описание проектируемых функций в формате схемы

На рисунке 2.2 представлена схема работы функции `get_coach_specializations`, которая получает список специализаций тренера.

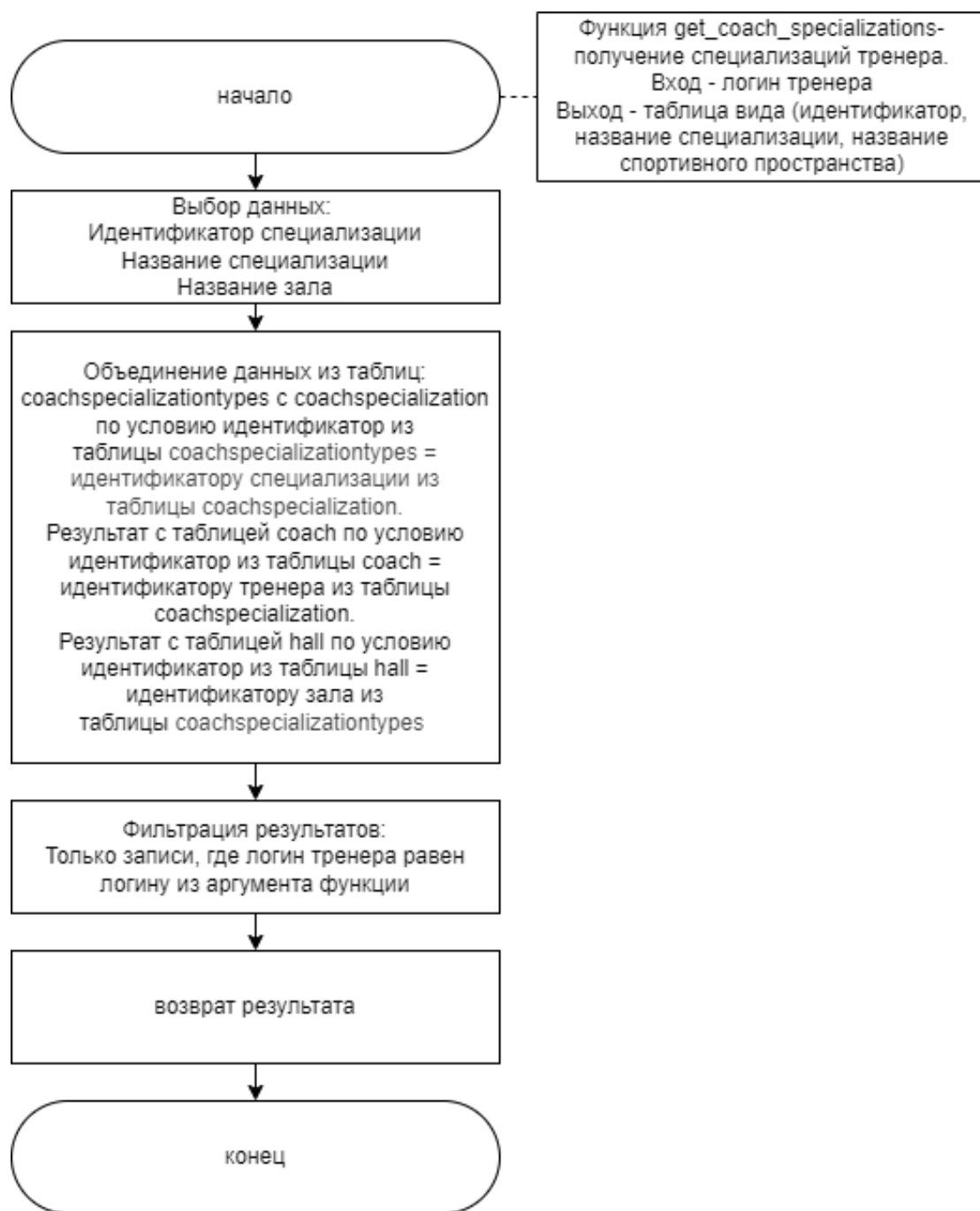


Рисунок 2.2 – Схема алгоритма функции `get_coach_specializations`

На рисунке 2.3 представлена схема работы функции `show_available_events`, которая получает список доступных для записи мероприятий.

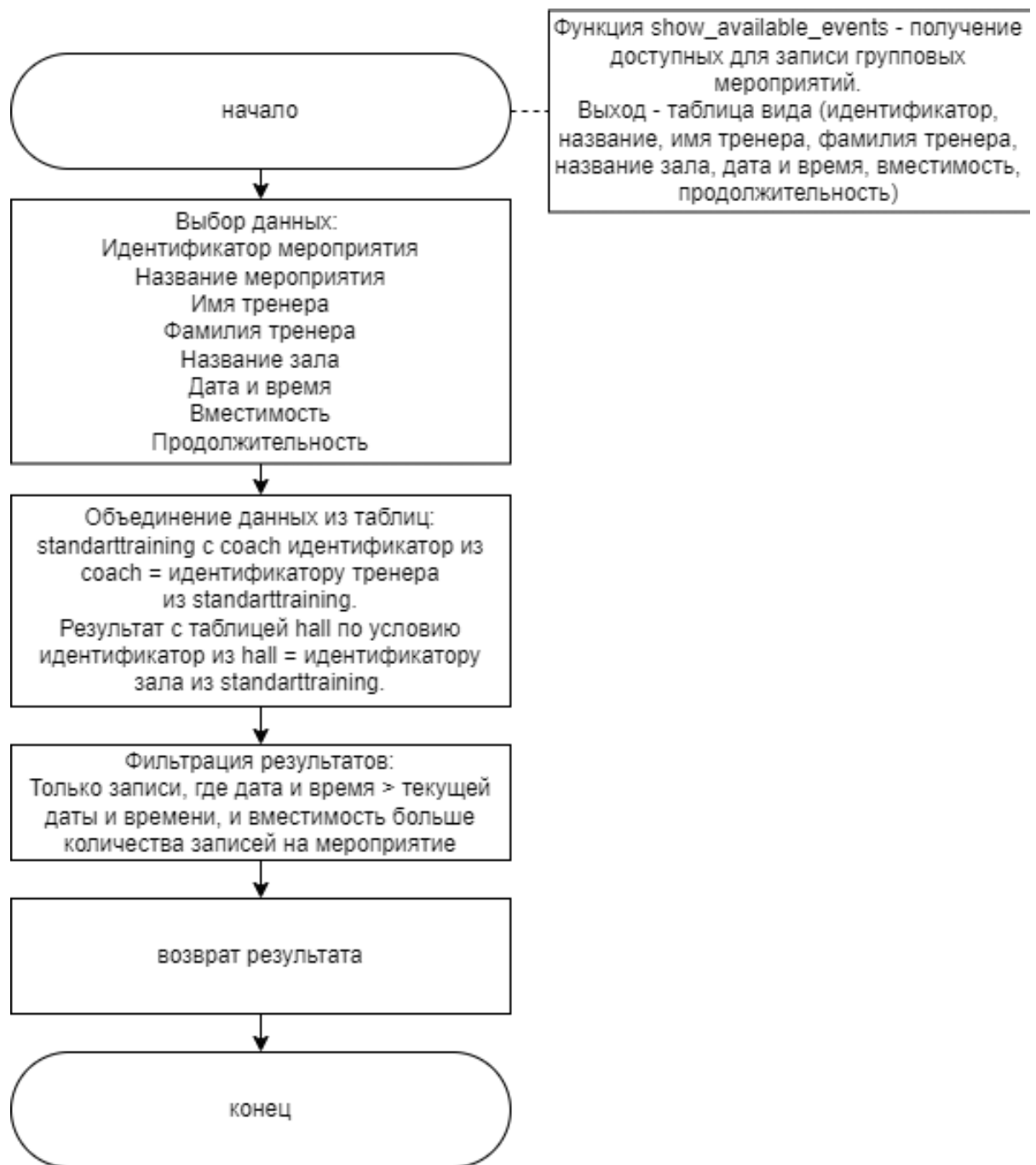


Рисунок 2.3 – Схема алгоритма функции show_available_events

На рисунке 2.4 представлена схема работы функции register_client_for_training, которая выполняет регистрацию клиента на мероприятие.

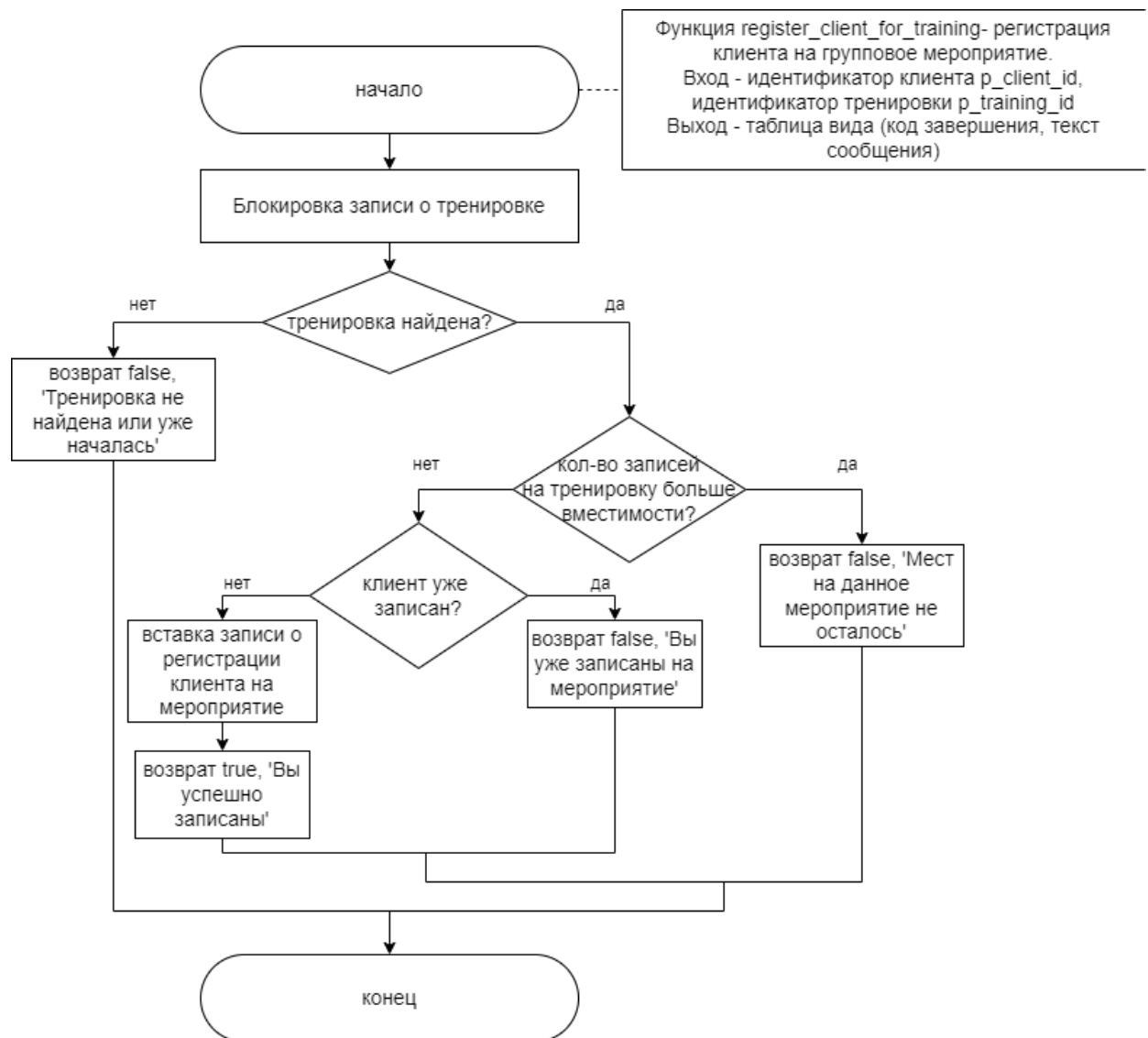


Рисунок 2.4 – Схема алгоритма функции register_client_for_training

2.5 Описание проектируемой ролевой модели на уровне базы данных

Ролевая модель состоит из трёх ролей: клиента, тренера и администратора.

1. Администратор имеет следующие возможности:

- Просмотр, добавление и удаление данных в таблице ClientDynamicTypes;
- Просмотр данных в таблице Hall;
- Просмотр данных в таблице IndividualTraining;
- Просмотр, добавление и изменение данных в таблице CoachSchedule;

- Просмотр данных в таблице CoachSpecializationTypes;
- Просмотр, добавление и удаление данных в таблице CoachSpecialization;
- Просмотр, добавление и изменение данных в таблице HallSchedule;
- Просмотр, добавление и изменение данных в таблице Client;
- Просмотр, добавление и изменение данных в таблице Coach;
- Просмотр, добавление и изменение данных в таблице Admin;
- Полный доступ (просмотр, удаление, добавление, изменение) к таблице StandartTraining.

2. Клиент имеет следующие возможности:

- Просмотр данных в таблице ClientDynamicTypes;
- Просмотр, добавление и удаление данных в таблице ClientDynamic;
- Просмотр и изменение данных в таблице StandartTraining;
- Просмотр данных в таблице Hall;
- Просмотр данных в таблице Coach;
- Просмотр и удаление данных в таблице IndividualTraining;
- Просмотр, добавление и удаление данных в таблице TrainingSigns.

3. Тренер имеет следующие возможности:

- Просмотр, изменение и добавление данных в таблицу CoachSchedule;
- Просмотр данных в таблице StandartTraining;
- Просмотр данных в таблице Admin;
- Просмотр данных в таблице Coach;
- Просмотр данных в таблице Hall;
- Полный доступ (просмотр, удаление, добавление, изменение) к таблице IndividualTraining;
- Просмотр данных в таблице Client;
- Просмотр данных в таблице CoachSpecializationTypes;
- Просмотр данных в таблице CoachSpecialization.

3 Технологическая часть

3.1 Средства реализации

Для написания программы был выбран язык TypeScript [6]. Такой выбор был обусловлен следующим образом:

- В TypeScript имеется встроенная поддержка всех парадигм объектно-ориентированного программирования, что позволяет создать гибкую и легко расширяемую программу;
- TypeScript позволяет реализовать принцип инверсии зависимостей [7];
- TypeScript имеет необходимый функционал для управления базами данных;
- TypeScript обладает строгой типизацией, что необходимо при создании больших приложений.

Для реализации принципа инверсии зависимостей была использована библиотека Inversify [8]. Для хранения паролей в защищённом виде была использована библиотека bcrypt [9].

В качестве среды разработки была выбрана IDE WebStorm [10]. Она имеет встроенный многофункциональный отладчик, возможность просмотра и редактирования содержимого таблиц баз данных, подсветку синтаксиса, базовые возможности анализа кода, автодополнение кода и бесплатна для некоммерческого использования.

Приложение имеет монолитную архитектуру, что упрощает развёртывание и тестирование кода.

3.2 Структура классов

Классы программы делятся на 3 основных слоя, как показано на рисунке 3.1



Рисунок 3.1 – Верхнеуровневое разбиение на компоненты

1. Data Access – набор классов, представленных паттерном репозиторий, обращающиеся к базе данных;
2. Business Logic – набор классов, реализующий логику работы приложения;
3. Console User Interface – набор классов, реализующих работу с пользователем, общающиеся далее к слою бизнес-логики.

Слой доступа к данным представлен следующими классами:

1. AdminRepository – управление администраторами;
2. ClientDynamicRepository – управление персональными статистиками;
3. ClientRepository – управление клиентами;
4. CoachRepository – управление тренерами;
5. CoachScheduleRepository – управление расписанием тренеров;
6. EventsRepository – управление групповыми мероприятиями;
7. HallRepository – управление спортивными пространствами;
8. PersonalTrainingRepository – управление персональными тренировками;
9. PersonRepository – общие методы управления клиентами, тренерами, администраторами.

3.3 Выбор системы управления базами данных

Для анализа были выбраны наиболее распространённые системы управления базами данных, такие как:

- PostgreSQL [11];
- MySQL [12];
- SQLite [13];
- Microsoft SQL Server [14].

PostgreSQL – система управления объектно-реляционной базой данных с открытым исходным кодом и полной поддержкой SQL. Стабильно получает обновления и является кроссплатформенной. Имеет расширяемую систему встроенных языков программирования, возможность создавать собственные функции, триггеры, процедуры и индексы. Полностью отвечает требованиям ACID.

В MySQL делается упор на высокую скорость чтения, что делает её незаменимой для создания Web-приложений. Однако может быть не так эффективна в отличие от PostgreSQL в сложных аналитических запросах. MySQL имеет открытый исходный код, работает практически на всех основных платформах.

SQLite – компактная встраиваемая система управления базой данных. Имеет ограниченную поддержку SQL синтаксиса, чаще всего используется в мобильной разработке, десктопных приложениях, браузерах, когда не надо обмениваться данными с сервером. Среди других особенностей архитектуры можно выделить малый размер памяти, требуемый для установки SQLite, и наличие динамической типизации.

Microsoft SQL Server подходит для создания проектов любого размера. Имеет собственный язык запросов Transact-SQL и гибкие инструменты администрирования. Однако обладает рядом недостатков, среди которых выделяется ограниченная совместимость с операционными системами, отличными от Windows, и необходимость выделения больших аппаратных ресурсов.

Выбор системы управления базами данных был обусловлен следующими критериями:

1. наличие опыта работы;
2. бесплатное распространение;
3. наличие подробной документации;
4. поддержка SQL в достаточном объёме.

В таблице 3.1 приведены результаты анализа.

Таблица 3.1 – Результаты анализа существующих конкурентов

Номер критерия	PostgreSQL	MySQL	SQLite	Microsoft SQL Server
1	+	-	+	-
2	+	+	+	-
3	+	-	+	+
4	+	+	-	+

По итогам анализа был выбран PostgreSQL, так как он отвечает всем вышеизложенным критериям.

Сценарии создания базы данных и наложения ограничений приведены в приложениях А и В соответственно.

3.4 Классы эквивалентности

3.4.1 Классы эквивалентности для функции `get_coach_specializations`

Исходные записи в таблицах базы данных представлены в таблицах 3.2, 3.3, 3.4, 3.5:

Таблица 3.2 – Записи в таблице coach

ID	name	surname	login	password	is_participate
6	Андрей	Кондукторов	oltpzss4	пароль в защищённом виде	true
7	Егор	Пучков	egorpuchk	пароль в защищённом виде	true

Таблица 3.3 – Записи в таблице hall

ID	name
2	Йога-студия
3	Бассейн
1	Фитнес зал

Таблица 3.4 – Записи в таблице coachspecialization

ID	specid	coachid
1	1	6
12	3	6

Таблица 3.5 – Записи в таблице coachspecializationtypes

ID	name	hallid
1	Фитнес	1
2	Йога	2
3	Бокс	1
4	Плавание	3

Класс ввода несуществующего логина тренера

Вход: oltpzss2

Ожидаемый результат: ничего.

Класс наличия нескольких специализаций тренера

Вход: oltpzss4

Ожидаемый результат представлен в таблице 3.6:

Таблица 3.6 – Результат для класса наличия нескольких специализаций тренера

ID	Название	Зал проведения
1	Фитнес	Фитнес зал
3	Бокс	Фитнес зал

Класс отсутствия специализаций у тренера

Вход: egorpuchk

Ожидаемый результат: ничего.

3.4.2 Классы эквивалентности для функции `show_available_events`

Исходные записи в таблицах базы данных представлены в таблицах 3.7, 3.8:

Таблица 3.7 – Записи в таблице StandartTraining

ID	name	adminid	coachid	hallid	capacity	datetime	duration
2	Интенсивный бокс	2	6	1	10	2025-05-01 19:12:28	3
1	Йога	2	6	2	2	2025-06-01 10:48:59	1
3	Растяжка	2	6	1	15	2025-07-29 19:17:36	1

Таблица 3.8 – Записи в таблице TrainingSigns

ID	clientid	stdtrainingid
1	1	1
2	2	1

Класс отсутствия доступных тренировок из-за того, что они уже прошли

Вход: Время, когда клиент обращается к функции: 2025-08-01 10:48:59.
Ожидаемый результат: ничего.

Класс доступных тренировок, с отсечением прошедших и тех, на которые закончились места

Вход: Время, когда клиент обращается к функции: 2025-05-30 10:00:00.
Ожидаемый результат представлен в таблице 3.9:

Таблица 3.9 – Результат работы функции show_available_events

ID	Название	Имя тренера	Фамилия тренера	Название зала	Дата	Вместимость	Продолжительность
3	Растяжка	Андрей	Кондукторов	Фитнес зал	2025-07-29 19:17:36	15	1

3.4.3 Классы эквивалентности для функции register_client_for_training

Класс неуспешной записи из-за идентификатора несуществующего мероприятия

Вход: идентификатор клиента: 4, идентификатор мероприятия: 150.
Ожидаемый результат: false, 'Тренировка не найдена или уже началась'.

Класс неуспешной записи из-за отсутствия мест на мероприятие

Вход: идентификатор клиента: 4, идентификатор мероприятия: 1.
Ожидаемый результат: false, 'Мест на данное мероприятие не осталось'.

Класс успешной записи на мероприятие

Вход: идентификатор клиента: 4, идентификатор мероприятия: 3.
Ожидаемый результат: true, 'Вы успешно записаны'.
Новое состояние таблицы TrainingSigns представлено в таблице 3.10.

Таблица 3.10 – Обновлённая таблица TrainingSigns

ID	clientid	stdtrainingid
1	1	1
2	2	1
3	4	3

Класс неуспешной повторной записи на мероприятие

Вход: идентификатор клиента: 4, идентификатор мероприятия: 1.

Ожидаемый результат: false, 'Вы уже записаны на это мероприятие'.

3.5 Разработанные функции

В листингах 3.1 и 3.2 приведена реализация функции регистрации клиента на групповое мероприятие.

Листинг 3.1 – Функция регистрации клиента на мероприятие. Часть 1

```
CREATE OR REPLACE FUNCTION register_client_for_training(  
    p_client_id INT,  
    p_training_id INT  
)  
    RETURNS TABLE(success BOOLEAN, message TEXT)  
    LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_capacity INT;  
    v_count INT;  
    v_already_registered BOOLEAN;  
BEGIN  
    SELECT capacity INTO v_capacity  
    FROM standarttraining  
    WHERE id = p_training_id AND datetime > now()  
    FOR UPDATE;  
  
    IF NOT FOUND THEN  
        RETURN QUERY SELECT false, 'Тренировка не найдена или  
            уже началась';  
        RETURN;  
    END IF;  
  
    SELECT COUNT(*) INTO v_count  
    FROM trainingsigns  
    WHERE stdtrainingid = p_training_id;
```

Листинг 3.2 – Функция регистрации клиента на мероприятие. Часть 2

```
SELECT EXISTS (
    SELECT 1 FROM trainingsigns
    WHERE clientid = p_client_id AND stdtrainingid =
        p_training_id
) INTO v_already_registered;

IF v_count >= v_capacity THEN
    RETURN QUERY SELECT false, 'Мест на данное мероприятие
        не осталось';
ELSIF v_already_registered THEN
    RETURN QUERY SELECT false, 'Вы уже записаны на это
        мероприятие';
ELSE
    INSERT INTO trainingsigns(clientid, stdtrainingid)
    VALUES (p_client_id, p_training_id);

    RETURN QUERY SELECT true, 'Вы успешно записаны';
END IF;
END;
$$;
```

В листингах 3.3 и 3.4 приведена реализация функции получения списка специализаций тренера.

Листинг 3.3 – Функция получения списка специализаций конкретного тренера. Часть 1

```
CREATE OR REPLACE FUNCTION get_coach_specializations(p_login
    VARCHAR)
    RETURNS TABLE(
        id INT,
        name VARCHAR,
        hall_name VARCHAR
    )
    LANGUAGE sql
AS $$
SELECT
    cst.id,
    cst.name,
    h.name AS hall_name
```

Листинг 3.4 – Функция получения списка специализаций конкретного тренера.
Часть 2

```
FROM
    coachspecializationtypes cst
        JOIN coachspecialization cs ON cst.id = cs.specid
        JOIN coach c ON c.id = cs.coachid
        JOIN hall h ON cst.hallid = h.id
WHERE c.login = p_login;
$$;
```

В листингах 3.5 и 3.6 приведена реализация функции получения списка доступных групповых мероприятий.

Листинг 3.5 – Функция получения списка доступных групповых мероприятий.
Часть 1

```
CREATE OR REPLACE FUNCTION show_available_events()
    RETURNS TABLE (
        id INT,
        name VARCHAR,
        coach_name VARCHAR,
        coach_surname VARCHAR,
        hall_name VARCHAR,
        datetime TIMESTAMP,
        capacity INT,
        duration INT
    )
    LANGUAGE sql
AS $$
SELECT
    st.id,
    st.name,
    c.name as coach_name,
    c.surname as coach_surname,
    h.name as hall_name,
    st.datetime,
    st.capacity,
    st.duration
```

Листинг 3.6 – Функция получения списка доступных групповых мероприятий.
Часть 2

```
FROM standarttraining st
      JOIN coach c ON c.id = st.coachid
      JOIN hall h ON h.id = st.hallid
WHERE st.datetime >= NOW()
      AND st.capacity > (
      SELECT COUNT(ts.id)
      FROM trainingsigns ts
      WHERE ts.stdtrainingid = st.id
      )
ORDER BY st.datetime;
$$;
```

3.6 Разработанная ролевая модель

В листингах 3.7, 3.8 и 3.9 приведена реализация ролевой модели на уровне базы данных.

Листинг 3.7 – Ролевая модель. Часть 1

```
-- =====
-- Создание ролей приложения
-- =====

CREATE ROLE app_client NOLOGIN;
CREATE ROLE app_coach NOLOGIN;
CREATE ROLE app_admin NOLOGIN;

-- Базовые разрешения для всех ролей
GRANT CONNECT ON DATABASE postgres TO app_client, app_coach,
      app_admin;
GRANT USAGE ON SCHEMA public TO app_client, app_coach, app_admin;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO
      app_client, app_coach, app_admin;
```

Листинг 3.8 – Ролевая модель. Часть 2

```
-- =====
-- Права администратора
-- =====

GRANT SELECT, INSERT, UPDATE, DELETE ON
    standarttraining
TO app_admin;

GRANT SELECT, INSERT, DELETE ON
    clientdynamictypes,
    coachspecialization
TO app_admin;

GRANT SELECT, INSERT, UPDATE ON
    coachschedule,
    hallschedule
TO app_admin;

GRANT SELECT ON
    hall,
    individualtraining,
    coachspecializationtypes
TO app_admin;

GRANT SELECT, INSERT, UPDATE ON
    client,
    coach,
    admin
TO app_admin;

-- =====
-- Права клиента
-- =====

GRANT SELECT, INSERT, DELETE ON
    clientdynamic,
    trainingsigns
TO app_client;
```

Листинг 3.9 – Ролевая модель. Часть 3

```
GRANT SELECT, UPDATE ON
    standarttraining
TO app_client;

GRANT SELECT, DELETE ON
    individualtraining
TO app_client;

GRANT SELECT ON
    clientdynamictypes,
    hall,
    coach
TO app_client;

-- =====
-- Права тренера
-- =====

GRANT SELECT, INSERT, UPDATE, DELETE ON
    individualtraining
TO app_coach;

GRANT SELECT, INSERT, UPDATE ON
    coachschedule
TO app_coach;

GRANT SELECT ON
    standarttraining,
    admin,
    coach,
    hall,
    client,
    coachspecializationtypes,
    coachspecialization
TO app_coach;
```


3.7 Демонстрация работы программы

При запуске пользователя встречает меню с вариантами входа в аккаунт:

```
C:\Users\User\Desktop\uni\DB\app>node -r module-alias/register -r tsconfig-paths/register dist/index.js
```

ID	Название
0	Выйти
1	Войти как клиент
2	Войти как тренер
3	Войти как администратор

Введите номер действия:

Рисунок 3.2 – Начальное меню приложения

При входе в качестве клиента, пользователю становится доступен следующий набор действий:

ID	Название
0	Выйти
1	Список видов индивидуальных статистик
2	Добавить запись в статистику
3	Просмотреть данные конкретной статистики
4	Удалить запись в статистике
5	Список доступных мероприятий
6	Зарегистрироваться на мероприятие
7	Отменить регистрацию на мероприятие
8	Список записей
9	Отменить персональную тренировку

Введите номер действия:

Рисунок 3.3 – Меню программы клиента спортивного зала

При входе в качестве тренера, пользователю становится доступен следующий набор действий:

ID	Название
0	Выйти
1	Мои рабочие часы
2	Заполнить рабочие часы на конкретную дату
3	Заполнить рабочие часы на неделю
4	Список моих тренировок
5	Назначить индивидуальную тренировку
6	Отменить персональную тренировку
7	Перенести персональную тренировку
8	Добавить информацию о персональной тренировке
9	Изменить продолжительность персональной тренировки
10	Список доступных мне залов для проведения персональных тренировок

Введите номер действия:

Рисунок 3.4 – Меню программы тренера спортивного зала

При входе в качестве администратора, пользователю становится доступен следующий набор действий:

ID	Название
0	Выйти
1	Список видов индивидуальных статистик клиентов
2	Добавить вид индивидуальной статистики клиентов
3	Удалить вид индивидуальной статистики клиентов
4	Список залов
5	Расписание конкретного зала
6	Выставить расписание зала на конкретную дату
7	Выставить расписание зала по умолчанию на период
8	Список активных клиентов
9	Список активных тренеров
10	Список активных администраторов
11	Добавить клиента
12	Добавить тренера
13	Добавить администратора
14	Удалить клиента
15	Удалить тренера
16	Удалить администратора
17	Список видов тренерских специализаций
18	Список специализаций конкретного тренера
19	Добавить специализацию тренеру
20	Удалить специализацию у тренера
21	Рабочие часы конкретного тренера
22	Заполнить рабочие часы тренеру на конкретную дату
23	Заполнить рабочие часы тренеру на неделю
24	Список групповых мероприятий
25	Создать групповое мероприятие
26	Редактировать групповое мероприятие
27	Удалить групповое мероприятие
28	Список тренировок конкретного тренера

Введите номер действия:

Рисунок 3.5 – Меню программы администратора спортивного зала

4 Исследовательская часть

4.1 Постановка исследования

Предметом исследования является влияние применения различных типов индексов в базе данных на время выполнения запросов к базе данных.

В исследовании задействованы следующие типы индексов [15]:

1. BTREE – механизм работы данного индекса построен на В-дереве. Используются для увеличения скорости работы запросов, использующих операторы сравнения;
2. HASH – такие индексы хранят хэш-код, полученный из значения индексируемого столбца. Используются только для запросов, с оператором сравнения на равенство;
3. GIN – эффективны для запросов, в которых используются компонентные значения, такие как массивы.

4.2 Технические характеристики

Приведены технические характеристики устройства, на котором производилось исследование:

- Процессор 11th Gen Intel(R) Core(TM) i7-11370H @ 3.30 ГГц;
- 4 физических и 8 логических ядер;
- Оперативная память 16 ГБайт;
- Система Windows 11 Home.

Исследование проводилось на ноутбуке. Во время исследования устройство было подключено к питанию, все фоновые процессы были завершены.

4.3 План исследования

В процессе исследования последовательно выполнялись запросы к базе данных. Для каждой размерности таблицы из списка [200, 500, 1000, 2000,

5000, 10000, 25000, 50000, 75000, 100000, 200000, 300000, 400000, 500000] проводилось по 10 замеров, далее считалось среднее время выполнения. Замер времени выполнения каждого запроса осуществлялся с помощью конструкции EXPLAIN ANALYZE [16]. Перед началом тестирования было выключено параллельное выполнение запросов. Сами запросы осуществлялись на таблицах StandartTraining – основная таблица базы данных со множеством полей, для которой создавались индексы и StandartTraining_noindex – её копией, без использования индексов. Для каждого теста таблицы заполнялись тестовыми данными. Также, после каждого замера проводилась очистка таблиц, и очищались устаревшие кортежи с помощью конструкции VACUUM ANALYZE [17].

В листинге 4.1 приведено создание индексов базы данных. BTREE индекс создаётся на колонку названия мероприятия, HASH создаётся на поле даты и времени, GIN создаётся на массив идентификаторов тренера и спортивного пространства.

Листинг 4.1 – Создание индексов

```
CREATE INDEX IF NOT EXISTS idx_stdtraining_name_btree
ON StandartTraining USING BTREE(name);

CREATE INDEX IF NOT EXISTS idx_stdtraining_datetime_hash
ON StandartTraining USING HASH(datetime);

CREATE INDEX IF NOT EXISTS idx_stdtraining_coach_hall_gin
ON StandartTraining USING GIN((array[CoachID, HallID]));
```

В листингах 4.2, 4.3, 4.4 приведены тестовые запросы, использующие BTREE, HASH, GIN индексы соответственно.

Листинг 4.2 – Запрос с использованием BTREE-индекса

```
EXPLAIN ANALYZE SELECT * FROM StandartTraining
WHERE name = 'Йога для начинающих';
```

Листинг 4.3 – Запрос с использованием HASH-индекса

```
EXPLAIN ANALYZE SELECT * FROM StandartTraining
WHERE datetime = '2025-06-01 10:00:00';
```

Листинг 4.4 – Запрос с использованием GIN-индекса

```
EXPLAIN ANALYZE SELECT * FROM StandartTraining
WHERE array[CoachID, HallID] @> array[2, 3];
```

В листингах 4.5, 4.6, 4.7 приведены тестовые запросы, эквивалентные запросам в листингах 4.2, 4.3, 4.4 соответственно, но без использования индексов.

Листинг 4.5 – Запрос без использования BTREE-индекса

```
EXPLAIN ANALYZE SELECT * FROM StandartTraining_noindex
WHERE name = 'Йога для начинающих';
```

Листинг 4.6 – Запрос без использования HASH-индекса

```
EXPLAIN ANALYZE SELECT * FROM StandartTraining_noindex
WHERE datetime = '2025-06-01 10:00:00';
```

Листинг 4.7 – Запрос без использования GIN-индекса

```
EXPLAIN ANALYZE SELECT * FROM StandartTraining_noindex
WHERE CoachID = 2 AND HallID = 3;
```

4.4 Результаты исследования

По результатам исследования были получены значения времени выполнения запросов, приведённые в таблицах 4.1, 4.2, 4.3.

Таблица 4.1 – Время выполнения запросов с BTREE индексом и без

Количество записей в таблице	Без индекса(мс)	С индексом(мс)
200	0.0232	0.0199
500	0.0387	0.0463
1000	0.0638	0.0527
2000	0.1180	0.0831
5000	0.2762	0.1182
10000	0.5252	0.2101
25000	1.4927	0.4418
50000	2.5391	0.9598
75000	3.9915	1.3309
100000	5.4074	1.7083
200000	13.4504	3.5008
300000	24.8811	5.3168
400000	30.6230	7.2296
500000	37.5275	8.5517

Таблица 4.2 – Время выполнения запросов с HASH индексом и без

Количество записей в таблице	Без индекса(мс)	С индексом(мс)
200	0.0183	0.0184
500	0.0407	0.0153
1000	0.0629	0.0145
2000	0.1316	0.0154
5000	0.2900	0.0135
10000	0.5646	0.0133
25000	1.3692	0.0168
50000	2.9975	0.0165
75000	4.3463	0.0146
100000	5.5263	0.0169
200000	11.3494	0.0228
300000	41.7652	0.0319
400000	54.2973	0.0309
500000	47.1357	0.0407

Таблица 4.3 – Время выполнения запросов с GIN индексом и без

Количество записей в таблице	Без индекса(мс)	С индексом(мс)
200	0.0169	0.0368
500	0.0337	0.0841
1000	0.0572	0.0608
2000	0.1075	0.0993
5000	0.2567	0.2156
10000	0.5087	0.4218
25000	1.2448	1.0174
50000	2.6809	2.1434
75000	4.0070	3.2908
100000	5.3215	4.3646
200000	11.1057	8.8471
300000	16.1611	13.1854
400000	30.7479	18.0912
500000	38.5459	21.9800

На основе таблиц 4.1, 4.2, 4.3 были построены графики 4.1, 4.2, 4.3 соответственно.

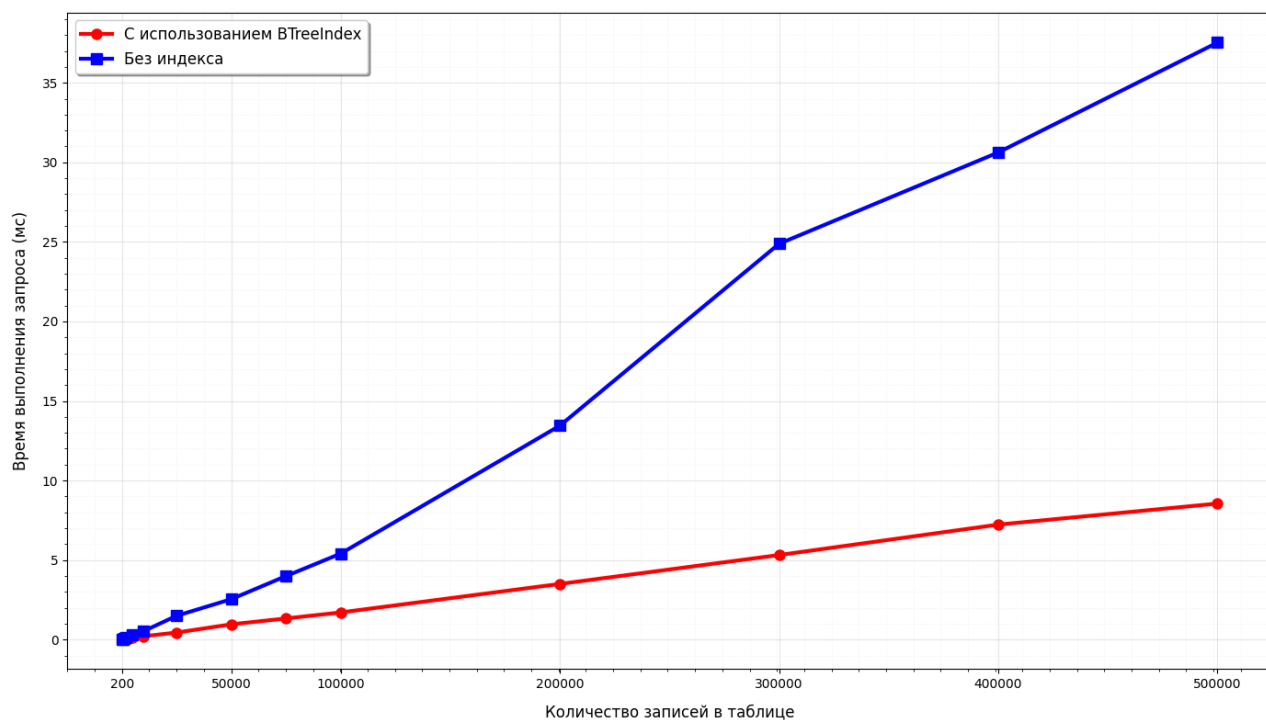


Рисунок 4.1 – График времени выполнения запросов от количество записей в таблице с использованием BТRЕЕ-индекса и без него

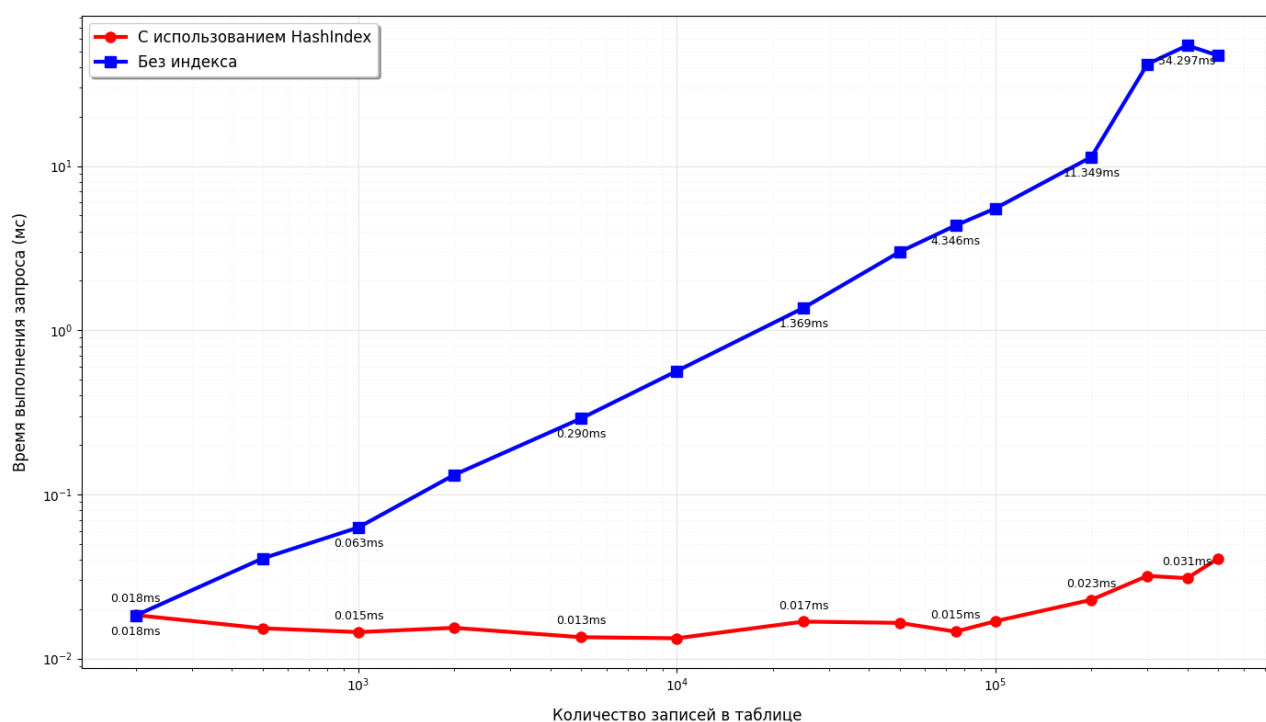


Рисунок 4.2 – График времени выполнения запросов от количество записей в таблице с использованием HASH-индекса и без него

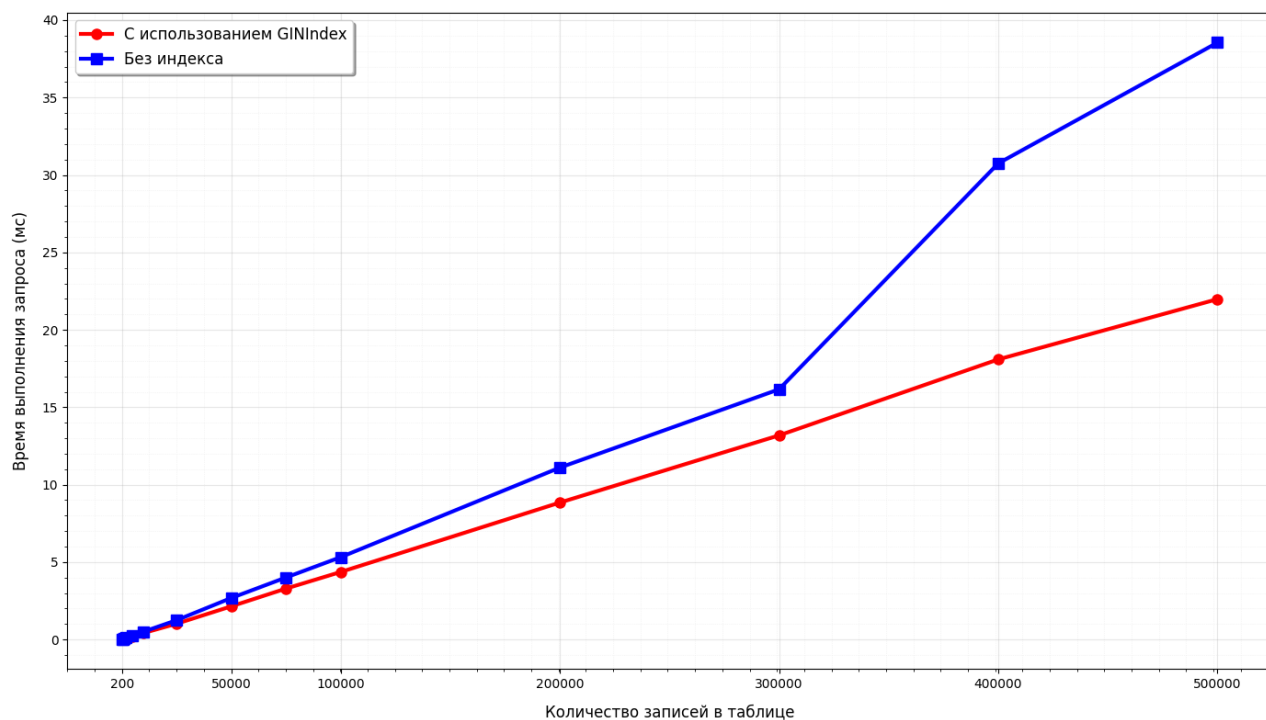


Рисунок 4.3 – График времени выполнения запросов от количество записей в таблице с использованием GIN-индекса и без него

4.5 Вывод

Использование индексов значительно уменьшает время выполнения запросов, что особенно заметно на больших таблицах.

ЗАКЛЮЧЕНИЕ

Цель курсовой работы была достигнута: разработано программное обеспечение с использованием базы данных для автоматизации деятельности спортивного зала

Решены все поставленные задачи:

- сформулированы ограничения предметной области;
- проанализированы существующие решения;
- спроектирована база данных;
- спроектированы способы взаимодействия с базой данных;
- разработана база данных и приложение доступа к базе данных;
- проведено исследование разработанной базы данных.

В ходе исследования было обнаружено, что применение индексов существенно уменьшает время выполнения запросов к базе данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. 1С:Фитнес-клуб – Программа для автоматизации и управления фитнес-клубом. [Электронный ресурс]. — Режим доступа: <https://www.fitness1c.ru/?ysclid=mb5exmnlvu385649358> (дата обращения: 10.05.2025).
2. Mobifitness – Система управления фитнес-клубом, спортивной студией, школой танцев или йоги [Электронный ресурс]. — Режим доступа: <https://mobifitness.ru/?ysclid=mb5f2q8xj7883010596> (дата обращения: 10.05.2025).
3. Программа для управления фитнес-бизнесом «всё в одном» [Электронный ресурс]. — Режим доступа: https://fitbase.io/?utm_source=yadirect&utm_medium=cpc&utm_campaign=pq|115512617|Brend_Poisk_Moskva&utm_content=16625141225&utm_term=FitBase&yclid=17312283845880446975 (дата обращения: 10.05.2025).
4. Sport Priority – Комплексное IT-сопровождение фитнес-клубов. [Электронный ресурс]. — Режим доступа: <https://sportpriority.com> (дата обращения: 10.05.2025).
5. О.Е. Аврунев, В.М. Стасышин — Модели баз данных – Новосибирск: Изд-во НГТУ, 2018. – 124 с – ISBN 978-5-7782-3749-0.
6. TypeScript is JavaScript with syntax for types. [Электронный ресурс]. — Режим доступа: <https://www.typescriptlang.org/> (дата обращения: 22.12.2024).
7. Dependency Injection Pattern in TypeScript: Enhancing Flexibility and Testability. [Электронный ресурс]. — Режим доступа: <https://softwarepatternslexicon.com/patterns-ts/4/7/> (дата обращения: 17.04.2025).
8. InversifyJS. A powerful and lightweight inversion of control container for JavaScript & Node.js apps powered by TypeScript [Электронный ресурс]. — Режим доступа: <https://inversify.io/> (дата обращения: 17.04.2025).
9. NPM – node.bcrypt.js [Электронный ресурс]. — Режим доступа: <https://www.npmjs.com/package/bcrypt?activeTab=readme> (дата обращения: 25.04.2025).

10. WebStorm – The JavaScript and TypeScript IDE [Электронный ресурс]. — Режим доступа: <https://www.jetbrains.com/webstorm/?ysclid=mb5gtxfxiy77118398> (дата обращения: 10.04.2025).
11. PostgreSQL: The World’s Most Advanced Open Source Relational Database [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/> (дата обращения: 14.04.2025).
12. MySQL: The world’s most popular open source database [Электронный ресурс]. — Режим доступа: <https://www.mysql.com/products/> (дата обращения: 14.04.2025).
13. What Is SQLite? [Электронный ресурс]. — Режим доступа: <https://www.sqlite.org/> (дата обращения: 14.04.2025).
14. Документация по Microsoft SQL [Электронный ресурс]. — Режим доступа: https://learn.microsoft.com/ru-ru/SQL/?view=sql-server-ver16&WT.mc_id=akams (дата обращения: 14.04.2025).
15. Postgres Documentation – Indexes Introduction [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/current/indexes-intro.html> (дата обращения: 10.05.2025).
16. Postgres Documentation – Using EXPLAIN. [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/current/using-explain.html> (дата обращения: 10.05.2025).
17. Postgres Documentation – VACUUM [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/current/sql-vacuum.html> (дата обращения: 10.05.2025).

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе состоит из 13 слайдов.

ПРИЛОЖЕНИЕ Б

Сценарий создания базы данных

Листинг Б.1 – Сценарий создания базы данных. Часть 1

```
CREATE TABLE IF NOT EXISTS Coach (  
    ID serial PRIMARY KEY,  
    name varchar(20),  
    surname varchar(20),  
    login varchar(10),  
    password varchar(255),  
    is_participate boolean  
);  
  
CREATE TABLE IF NOT EXISTS CoachSpecializationTypes (  
    ID serial PRIMARY KEY,  
    name varchar(20),  
    HallID int  
);  
  
CREATE TABLE IF NOT EXISTS CoachSpecialization (  
    ID serial PRIMARY KEY,  
    SpecID int,  
    CoachID int  
);  
  
CREATE TABLE IF NOT EXISTS CoachSchedule (  
    ID serial PRIMARY KEY,  
    CoachID int,  
    date date,  
    start time,  
    stop time  
);  
  
CREATE TABLE IF NOT EXISTS Client (  
    ID serial PRIMARY KEY,  
    name varchar(20),  
    surname varchar(20),  
    login varchar(10),  
    password varchar(255),  
    is_participate boolean  
);
```

Листинг Б.2 – Сценарий создания базы данных. Часть 2

```
CREATE TABLE IF NOT EXISTS ClientDynamicTypes (  
    ID serial PRIMARY KEY,  
    name varchar(30),  
    unit varchar(20)  
);  
  
CREATE TABLE IF NOT EXISTS ClientDynamic (  
    ID serial PRIMARY KEY,  
    ClientID int,  
    DynamicTypeID int,  
    value float,  
    date Date  
);  
  
CREATE TABLE IF NOT EXISTS Admin (  
    ID serial PRIMARY KEY,  
    name varchar(20),  
    surname varchar(20),  
    login varchar(10),  
    password varchar(255),  
    is_participate boolean  
);  
  
CREATE TABLE IF NOT EXISTS StandartTraining (  
    ID serial PRIMARY KEY,  
    name varchar(50),  
    AdminID int,  
    CoachID int,  
    HallID int,  
    capacity int,  
    datetime timestamp,  
    duration float  
);  
  
CREATE TABLE IF NOT EXISTS Hall (  
    ID serial PRIMARY KEY,  
    name varchar(30)  
);
```


Листинг Б.3 – Сценарий создания базы данных. Часть 3

```
CREATE TABLE IF NOT EXISTS HallSchedule (  
    ID serial PRIMARY KEY,  
    HallID int,  
    date date,  
    start time,  
    stop time  
);  
  
CREATE TABLE IF NOT EXISTS TrainingSigns (  
    ID serial PRIMARY KEY,  
    ClientID int,  
    StdTrainingID int  
);  
  
CREATE TABLE IF NOT EXISTS IndividualTraining (  
    ID serial PRIMARY KEY,  
    CoachID int,  
    ClientID int,  
    HallID int,  
    datetime timestamp,  
    duration float,  
    notes text  
);
```

ПРИЛОЖЕНИЕ В

Сценарий наложения ограничений на таблицы базы данных

Листинг В.1 – Сценарий наложения ограничений на таблицы. Часть 1

```
ALTER TABLE Coach
    ALTER COLUMN name SET NOT NULL,
    ADD CHECK (name ~ '^[A-Za-zA-Яа-я]+$'),

    ALTER COLUMN surname SET NOT NULL,
    ADD CHECK (surname ~ '^[A-Za-zA-Яа-я]+$'),

    ALTER COLUMN login SET NOT NULL,
    ADD CHECK (login ~ '^[A-Za-z0-9]+$'),

    ALTER COLUMN password SET NOT NULL,
    ADD CHECK (length(password) >= 8),

    ADD UNIQUE (login),
    ADD CHECK (length(login) BETWEEN 8 AND 10),

    ALTER COLUMN is_participate SET DEFAULT true;

ALTER TABLE CoachSpecializationTypes
    ALTER COLUMN name SET NOT NULL,
    ADD FOREIGN KEY (HallID) REFERENCES Hall(ID),
    ADD UNIQUE (name);

ALTER TABLE CoachSpecialization
    ADD FOREIGN KEY (SpecID) REFERENCES
        CoachSpecializationTypes(ID),
    ADD FOREIGN KEY (CoachID) REFERENCES Coach(ID),
    ADD CONSTRAINT unique_coachid_specid UNIQUE (CoachID,
        SpecID);
```

Листинг В.2 – Сценарий наложения ограничений на таблицы. Часть 2

```
ALTER TABLE CoachSchedule
    ADD FOREIGN KEY (CoachID) REFERENCES Coach(ID),
    ALTER COLUMN date SET NOT NULL,
    ALTER COLUMN start SET NOT NULL,
    ALTER COLUMN stop SET NOT NULL,
    ADD CONSTRAINT unique_coachid_date UNIQUE (CoachID, date);

ALTER TABLE Client
    ALTER COLUMN name SET NOT NULL,
    ADD CHECK (name ~ '^[A-Za-zА-Яа-я]+$'),

    ALTER COLUMN surname SET NOT NULL,
    ADD CHECK (surname ~ '^[A-Za-zА-Яа-я]+$'),

    ALTER COLUMN login SET NOT NULL,
    ADD CHECK (login ~ '^[A-Za-z0-9]+$'),

    ALTER COLUMN password SET NOT NULL,
    ADD CHECK (length(password) >= 8),

    ADD UNIQUE (login),
    ADD CHECK (length(login) BETWEEN 8 AND 10),

    ALTER COLUMN is_participate SET DEFAULT true;

ALTER TABLE ClientDynamicTypes
    ALTER COLUMN name SET NOT NULL,
    ALTER COLUMN unit SET NOT NULL,
    ADD UNIQUE (name);

ALTER TABLE ClientDynamic
    ADD FOREIGN KEY (ClientID) REFERENCES Client(ID),
    ADD FOREIGN KEY (DynamicTypeID) REFERENCES
        ClientDynamicTypes(ID),
    ALTER COLUMN value SET NOT NULL,
    ALTER COLUMN date SET NOT NULL,
    ADD CONSTRAINT unique_dynamictypeid_date_clientid UNIQUE
        (DynamicTypeID, date, ClientID);
```

Листинг В.3 – Сценарий наложения ограничений на таблицы. Часть 3

```
ALTER TABLE Admin
    ALTER COLUMN name SET NOT NULL,
    ADD CHECK (name ~ '^[A-Za-zA-Яа-я]+$'),

    ALTER COLUMN surname SET NOT NULL,
    ADD CHECK (surname ~ '^[A-Za-zA-Яа-я]+$'),

    ALTER COLUMN login SET NOT NULL,
    ADD CHECK (login ~ '^[A-Za-z0-9]+$'),

    ALTER COLUMN password SET NOT NULL,
    ADD CHECK (length(password) >= 8),

    ADD UNIQUE (login),
    ADD CHECK (length(login) BETWEEN 8 AND 10),

    ALTER COLUMN is_participate SET DEFAULT true;

ALTER TABLE StandartTraining
    ALTER COLUMN name SET NOT NULL,
    ADD CHECK (name ~ '^[A-Za-zA-Яа-я]+(?: [A-Za-zA-Яа-я]+)*$'
        AND name !~ '^+$'),

    ADD FOREIGN KEY (AdminID) REFERENCES Admin(ID),
    ADD FOREIGN KEY (CoachID) REFERENCES Coach(ID),
    ADD FOREIGN KEY (HallID) REFERENCES Hall(ID),

    ALTER COLUMN capacity SET NOT NULL,
    ADD CHECK (capacity > 0),

    ALTER COLUMN datetime SET NOT NULL,

    ALTER COLUMN duration SET NOT NULL,
    ADD CHECK (duration > 0);
```

Листинг В.4 – Сценарий наложения ограничений на таблицы. Часть 4

```
ALTER TABLE Hall
    ALTER COLUMN name SET NOT NULL,
    ADD CHECK (name ~ '[A-Za-zA-Яa-я0-9\ -]+(?:[A-Za-zA-Яa-я0-9\ -]+)*$'
        AND name !~ '^+$'),
    ADD UNIQUE (name);

ALTER TABLE HallSchedule
    ADD FOREIGN KEY (HallID) REFERENCES Hall(ID),
    ALTER COLUMN date SET NOT NULL,
    ALTER COLUMN start SET NOT NULL,
    ALTER COLUMN stop SET NOT NULL,
    ADD CONSTRAINT unique_hallid_date UNIQUE (HallID, date);

ALTER TABLE TrainingSigns
    ADD FOREIGN KEY (ClientID) REFERENCES Client(ID),
    ADD FOREIGN KEY (StdTrainingID) REFERENCES
        StandartTraining(ID),
    ADD CONSTRAINT unique_clientid_stdtrainingid UNIQUE
        (ClientID, StdTrainingID);

ALTER TABLE IndividualTraining
    ADD FOREIGN KEY (CoachID) REFERENCES Coach(ID),
    ADD FOREIGN KEY (ClientID) REFERENCES Client(ID),
    ADD FOREIGN KEY (HallID) REFERENCES Hall(ID),

    ALTER COLUMN datetime SET NOT NULL,
    ALTER COLUMN duration SET NOT NULL,
    ADD CHECK (duration > 0),
    ADD CONSTRAINT unique_coachid_datetime UNIQUE (CoachID,
        datetime);
```